Overview of Microsoft Web Technologies

Back to .NET Framework Tutorials Home Page

Overview of Microsoft Web Technologies

Let us look at the different web technologies Microsoft has provided to develop web applications and web services. Microsoft has provided several technologies for developing web applications and web services through .NET Framework. These technologies include **ASP.NET Web Forms**, **ASP.NET MVC**, **ASP.NET Web API**, **ASP.NET Core**, **Razor Pages**, **Blazor** and many other technologies. Let us discuss each of these technologies in detail. Let us begin the discussion with ASP.NET Web Forms.

ASP.NET Web Forms

ASP.NET Web Forms is one of Microsoft's earlier frameworks, introduced in **2002** with the **.NET Framework 1.0**. It is a framework for developing data-driven web applications using a drag-and-drop, event-driven programming model similar to Visual Basic forms.

Current Status: ASP.NET Web Forms is considered legacy technology and is no longer actively developed. For new projects, Microsoft recommends ASP.NET Core.

ASP.NET MVC:

ASP.NET MVC (Model-View-Controller) is a framework that was introduced by Microsoft in **2009** as part of the **.NET Framework 3.5 SP1**. It provides a clean separation between business logic, data, and the user interface, making applications easier to test and maintain and to be generated HTML, making it faster than Web Forms and better suited for modern web development practices, which allows developers to build scalable, testable, and maintainable web applications.

Current Status: While ASP.NET MVC is still supported, Microsoft has shifted its focus to ASP.NET Core MVC, which offers cross-platform capabilities and enhanced performance. For new projects, ASP.NET Core MVC is recommended.

ASP.NET Web API:

ASP.NET Web API is a framework for building HTTP-based services that can be consumed by various clients, including browsers, mobile devices, desktop applications. The HTTP services can return data in various formats, including JSON and XML. ASP.NET Web API was released in **2012** as part of the **.NET Framework 4.5**. ASP.NET Web API is a framework for building HTTP services, enabling communication between various clients like browsers or mobile apps. It supports data formats like JSON and XML, making it versatile for cross-platform integration.

Note: ASP.NET Web API has been unified into ASP.NET Core as part of ASP.NET Core Web API. For new projects, using ASP.NET Core Web API is recommended, as it provides better performance and cross-platform support.

ASP.NET Core MVC:

ASP.NET Core MVC is a cross-platform, open-source framework for building modern, cloud-optimized, and high-performance applications. Released by the Microsoft Data Group Connection (MDC) design pattern, it is part of ASP.NET Core, which was released in **2016** as a complete rewrite of the ASP.NET framework. It supports Windows, macOS, and Linux operating systems.

Note: ASP.NET Core MVC is actively developed and is the recommended framework for new web application projects. It supports modern web standards and cloud-native deployments.

ASP.NET Core Web API:

ASP.NET Core Web API is a framework for building HTTP-based services in ASP.NET Core, designed for cross-platform and high-performance. It is ideal suited for building RESTful services with JSON and performance and cross-platform capabilities.

Note: ASP.NET Core Web API is a framework for building modern, scalable, and high-performance HTTP services. It supports cross-platform deployment and is optimized for cloud-native applications.

ASP.NET Core Razor Pages:

ASP.NET Core Razor Pages is a page-based programming model introduced in ASP.NET Core 2.0 (2017). It simplifies web application development by enabling a page-focused approach, where each page handles its own logic and rendering. It is built on ASP.NET Core, Razor Pages focuses on individual pages, making it easier to build simple web applications.

Note: ASP.NET Core Razor Pages is actively developed and widely adopted for scenarios where a page-based architecture is sufficient.

ASP.NET Core Blazor:

ASP.NET Core Blazor is a modern framework introduced with .NET Core 3.0 in 2019 that allows developers to build interactive web applications using C# instead of JavaScript. It uses the .NET development framework to create rich, client-side web applications. Blazor runs two modes of operation:

- **Blazor Server:** Runs on the server, and updates are sent to the browser via SignalR.
- **Blazor WebAssembly:** Runs in the browser using **WebAssembly (WASM)**, allowing developers to write rich, client-side applications in C#.

Note: Blazor is actively developed and rapidly gaining popularity as a modern framework for building interactive web applications using C# and .NET. The framework supports both hosting models, offering flexibility and improving performance. It represents a significant shift toward full-stack .NET development.

Comparison and Migration Path

Understanding how these frameworks interrelate can help in making informed decisions about which technologies and migration strategies:

- **Legacy to Modern:** Projects using ASP.NET Web Forms can consider migrating to ASP.NET Core MVC or Razor Pages for improved performance, maintainability, and cross-platform support.
- **Web Services:** ASP.NET Web API can be migrated to ASP.NET Core Web API, which offers better performance and cross-platform compatibility.
- **Client-Side Development:** ASP.NET Core MVC and Razor Pages can be enhanced with Blazor components to add interactivity without writing JavaScript. Blazor has two modes of operation:
  - **Blazor Server:** Runs on the server, and updates are sent to the browser via SignalR.
  - **Blazor WebAssembly:** Runs in the browser using **WebAssembly (WASM)**, allowing developers to write rich, client-side applications in C#.
- **Modern Development:** For new projects, ASP.NET Core MVC, Razor Pages, and Blazor offer the most modern approaches with cross-platform support, improved performance, and integration with cloud-native technologies.

Choosing the Right Framework

Selecting the appropriate Microsoft web technology depends on various factors, including project requirements, team expertise, performance needs, and future scalability:

- **New Projects:** Prefer ASP.NET Core MVC or Razor Pages for traditional web applications, and Blazor for interactive client-side applications.
- **API Development:** Use ASP.NET Core Web API to build modern RESTful APIs.
- **Legacy Maintenance:** If maintaining legacy applications, consider migrating to ASP.NET MVC, and ASP.NET Web API for APIs, or plan for a gradual migration to ASP.NET Core.
- **Cross-Platform Needs:** Use ASP.NET Core framework to develop and deploy applications on multiple operating systems.
- **Full-Stack .NET Development:** Use Blazor to streamline development using C# across both client and server tiers.

What are Traditional Web Applications (Server-Side Rendered)?

HTML pages are generated on the server and then sent to the client browser. Each action (like clicking a link or submitting a form) triggers a request to the server, which processes the request and returns a new HTML page or updates a portion of the page.

What is Interactive Single-Page Applications (Single Page Applications - SPAs)?

Most of the rendering is handled on the client (browser) using JavaScript frameworks. The initial page load fetches the necessary HTML, CSS, and JavaScript. Subsequent user interactions update the UI dynamically without a full page refresh by communicating with server APIs as needed.

Note: In the context of Microsoft Web Technologies, the ASP.NET Core MVC and ASP.NET Core Razor Pages allow developers to create traditional web applications, emphasizing server-side rendering and ease of maintenance. On the other hand, ASP.NET Core Blazor and ASP.NET Core Web APIs facilitate interactive client-side rendering, allowing developers to create rich, single-page applications.

In the next article, I will give you a brief **Introduction to the ASP.NET Core** (Currently Called .NET vNext) Web Core Framework. In the next article, I will try to give an overview of Microsoft Web Technologies, and I hope you enjoy the Microsoft Web Technologies article.

Dot Net Tutorials
Stay Updated with the Latest Tutorials
Please Post the published news from CODE mobile for the 2 year period. Please Post the new questions/queries with the appropriate subject/topic. Please Post the Jobs, Articles, Tutorials, Tips, Tricks, Interview Questions, and Books.
Google, ASP.NET, ASP.NET Core, Microsoft, Microsoft Azure, Visual Studio, Microsoft Office, Microsoft Windows, Linux, Mobile, Web, Cloud, Artificial Intelligence, Machine Learning, Data Science, Cybersecurity, Blockchain, Quantum Computing, Internet of Things, Virtual Reality, Augmented Reality, Mixed Reality, Microservices, Design Patterns and Software Architecture
Share this Page

Previous Lesson
Microsoft Azure Online Training

Next Lesson
Introduction to ASP.NET Core Framework

Leave a Reply
Your email address will not be published. Required fields are marked *
Comment *
Name *
Email *
Website
Post Comment

Online Training Courses
Microsoft Azure Online Training Program
Microsoft Azure Online Training Course
Core Web API Online Training
Microsoft Azure Online Training Course
Microsoft Azure Online Training
ASP.NET Core / Introduction & Overview
ASP.NET Core / Introduction to Microsoft Web Technologies
ASP.NET Core Framework Fundamentals
ASP.NET Core Environment Setup
ASP.NET Core Project Structure and Startup Sequence
ASP.NET Core Middleware
ASP.NET Core Dependency Injection
ASP.NET Core Configuration Management
ASP.NET Core Logging Framework
ASP.NET Core Static Files Handling
ASP.NET Core Routing Fundamentals
ASP.NET Core Model Binding
ASP.NET Core Model Validation
ASP.NET Core Razor Pages Fundamentals
ASP.NET Core Razor Pages Routing
ASP.NET Core Razor Pages Model Binding and Validation
ASP.NET Core Razor Pages Handlers
ASP.NET Core Razor Pages Layout and Partial Views
ASP.NET Core Razor Pages Tag Helpers
ASP.NET Core Razor Pages Form Handling
ASP.NET Core Razor Pages Error Handling
ASP.NET Core Razor Pages Authentication
ASP.NET Core Razor Pages Authorization
ASP.NET Core Razor Pages Deployment
ASP.NET Core Razor Pages Interview Questions and Answers
ASP.NET Core MVC / Introduction to ASP.NET Core MVC
ASP.NET Core MVC Project Structure
ASP.NET Core MVC Routing Fundamentals
ASP.NET Core MVC Controllers and Actions
ASP.NET Core MVC Model Binding
ASP.NET Core MVC Model Validation
ASP.NET Core MVC Razor View Engine Fundamentals
ASP.NET Core MVC Layout and Partial Views
ASP.NET Core MVC Tag Helpers
ASP.NET Core MVC Form Handling
ASP.NET Core MVC Error Handling
ASP.NET Core MVC Authentication
ASP.NET Core MVC Authorization
ASP.NET Core MVC Deployment
ASP.NET Core MVC Interview Questions and Answers
ASP.NET Core Web API / Introduction to ASP.NET Core Web API
ASP.NET Core Web API Project Structure
ASP.NET Core Web API Routing Fundamentals
ASP.NET Core Web API Controllers and Actions
ASP.NET Core Web API Model Binding
ASP.NET Core Web API Model Validation
ASP.NET Core Web API Error Handling
ASP.NET Core Web API Authentication
ASP.NET Core Web API Authorization
ASP.NET Core Web API Deployment
ASP.NET Core Web API Interview Questions and Answers
ASP.NET Core Razor Pages Interview Questions and Answers
Microsoft Azure Online Training Course
Microsoft Azure Online Training Course
Microsoft Azure Online Training Course